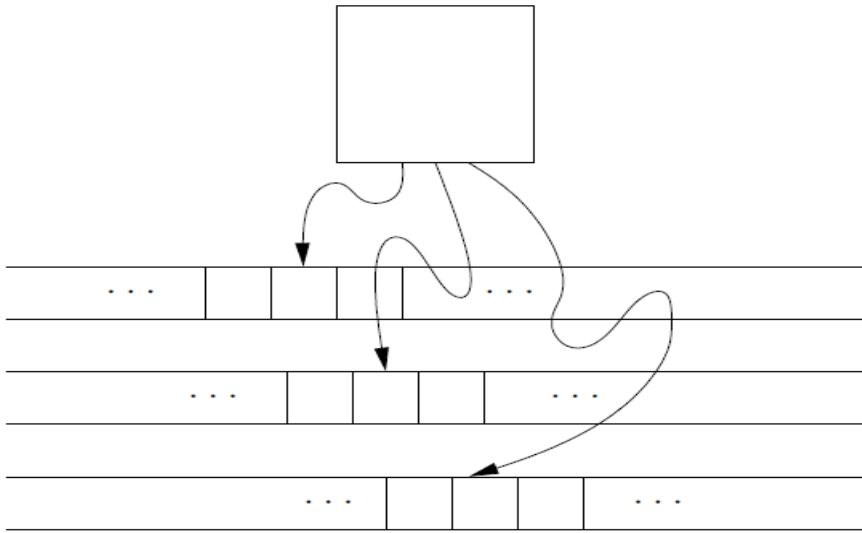


CSE 3204
Formal Language, Automata and Computability



Lecture 21
Turing Machine

Multitape Turing Machine



- Has a finite control (state) and some finite number of tapes.
- Each tape is divided into cells.
- Each cell contains a tape symbol from a finite set of tape symbols.

1. The input, a finite sequence of input symbols, is placed on the first tape.
2. All other cells of all the tapes hold the blank.
3. The finite control is in the initial state.
4. The head of the first tape is at the left end of the input.
5. All other tape heads are at some arbitrary cell. Since tapes other than the first tape are completely blank, it does not matter where the head is placed initially; all cells of these tapes “look” the same.

Multitape Turing Machine

A move of the multitape TM depends on the state and the symbol scanned by each of the tape heads. In one move, the multitape TM does the following:

1. The control enters a new state, which could be the same as the previous state.
 2. On each tape, a new tape symbol is written on the cell scanned. Any of these symbols may be the same as the symbol previously there.
 3. Each of the tape heads makes a move, which can be either left, right, or stationary. The heads move independently, so different heads may move in different directions, and some may not move at all.
- Accepts by entering a final state.

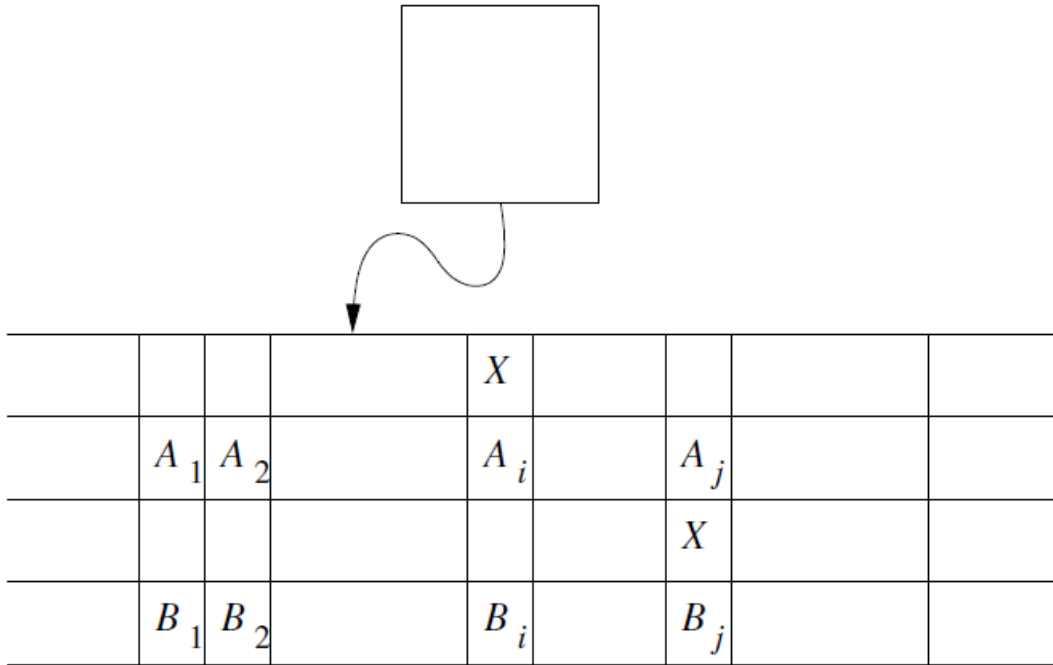
Equivalence of One-tape TM and Multitape TM

- RE Language is accepted by a one-tape TM.
- One-tape TM is a multitape TM
- Hence, multitape TM accepts all RE language.
- Theorem:

Every language accepted by a multitape TM is recursively enumerable.

Very easy proof. So read it by yourself from the book.

Proof of the Theorem



- **k**-tape TM **M**.
- TM **N** with one tape with **2k** tracks.
- Half of the tracks contains the content of the tapes of **M**.
- Remaining half contains a single marker in a cell marking the position of the corresponding tape head in **M**.
- $K=2$

Proof of the Theorem

- To simulate a move of M , N 's head must visit the k head markers of M .
- N stores the head count to its left, scanned symbol for each head, head's move and the corresponding state change in its finite control.
- N then revisits each of the head markers on its tape, changes the symbol in the track, moves the head left or right, if necessary. Finally, N changes the state of M .
- We select as N 's accepting states all those states that record M 's state as one of the accepting states of M .

Running Time of a TM

- The running time of TM **M** on input **w** is the number of steps that **M** makes before halting.
- If **M** does not halt on **w**, then the running time of **M** on **w** is infinite
- The time complexity of TM **M** is the function **T(n)** that is the maximum, over all inputs **w** of length n , of the running time of **M** on **w**.
- For Turing machines that do not halt on all inputs, **T(n)** may be infinite for some or even all n .

Theorem 8.10: The time taken by the one-tape TM N of Theorem 8.9 to simulate n moves of the k -tape TM M is $O(n^2)$.

PROOF: After n moves of M , the tape head markers cannot have separated by more than $2n$ cells. Thus, if N starts at the leftmost marker, it has to move no more than $2n$ cells right, to find all the head markers. It can then make an excursion leftward, changing the contents of the simulated tapes of M , and moving head markers left or right as needed. Doing so requires no more than $2n$ moves left, plus at most $2k$ moves to reverse direction and write a marker X in the cell to the right (in the case that a tape head of M moves right).

Thus, the number of moves by N needed to simulate one of the first n moves is no more than $4n + 2k$. Since k is a constant, independent of the number of moves simulated, this number of moves is $O(n)$. To simulate n moves requires no more than n times this amount, or $O(n^2)$. $\square$

Nondeterministic Turing Machine

A *nondeterministic* Turing machine (*NTM*) differs from the deterministic variety we have been studying by having a transition function δ such that for each state q and tape symbol X , $\delta(q, X)$ is a set of triples

$$\{(q_1, Y_1, D_1), (q_2, Y_2, D_2), \dots, (q_k, Y_k, D_k)\}$$

- A NTM $\mathbf{M}$ accepts an input $\mathbf{w}$ if there is any sequence of choices of move that leads from the initial ID with $\mathbf{w}$ as input to an ID with an accepting state.

Theorem 8.11: If M_N is a nondeterministic Turing machine, then there is a deterministic Turing machine M_D such that $L(M_N) = L(M_D)$.

Proof from the book.

Turing Machine with Semi-infinite Tapes

- TM's heads are allowed to move within the positions at and to the right of the initial head position.
- the tape is semi-infinite if there are no cells to the left of the initial head position. The TM is not allowed to move to the left of the initial head position.
- Construction of a semi-infinite TM simulating a TM with infinite tape.

X_0	X_1	X_2	...
*	X_{-1}	X_{-2}	...

Figure 8.19: A semi-infinite tape can simulate a two-way infinite tape

Turing Machine with Semi-infinite Tapes

Theorem 8.12: Every language accepted by a TM M_2 is also accepted by a TM M_1 with the following restrictions:

1. M_1 's head never moves left of its initial position.
2. M_1 never writes a blank.

Proof from the Book

- Section 8.4 and 8.5 of Chapter 8